

Pelican Panel on WSL2

Home Lab Project Documentation

Author	Bryan Sykes	Hostname	SykesHomeServer
Date	May 22, 2026	Location	Alexandria, VA
Project Type	Home Lab / Self-Hosted	Status	Operational
Panel	Pelican Panel + Wings	OS	Windows + WSL2 (Ubuntu 24)

1. Project Overview

This document covers the full setup of a self-hosted game server management panel running on Windows Subsystem for Linux 2 (WSL2). The stack includes Pelican Panel as the web-based management interface, Wings as the container daemon, native Docker Engine, and Pi-hole for network-wide ad blocking.

This was not a straightforward deployment. The core technical challenge — a WSL2 kernel limitation around cgroup v2 io.weight — required patching runc from source. This document captures every step including root cause analysis and the definitive fix, providing a reproducible path that did not exist in public documentation at the time of writing.

System Environment

Component	Details
Host OS	Windows with WSL2
WSL2 Distribution	Ubuntu 24
Server Hostname	SykesHomeServer
Windows Host IP	192.168.1.182
WSL2 Internal IP	172.26.11.55 (changes on reboot)
Panel URL	http://192.168.1.182:8080
Wings API Port	8081
SFTP Port	2022
Docker Engine	Native (not Docker Desktop)
runc Version	1.3.5 (patched)

2. Services & Status

Service	Status	Notes
Pi-hole	■ Running	Network-wide DNS ad blocker — 7.4M+ domains blocked
Pelican Panel	■ Running	http://192.168.1.182:8080
Wings Daemon	■ Running	systemd service — communicates with panel on port 8081
Minecraft (Paper)	■ Running	192.168.1.182:25565 — Java 25, Paper egg
Palworld	■ Running	192.168.1.182:8211 — v0.7.3.90464, Steam dedicated server

3. Pi-hole Setup

Pi-hole was deployed as a network-wide DNS sinkhole, blocking ads and trackers across all devices on the local network. It runs natively in WSL2 alongside the Pelican Panel stack.

Installation

```
curl -sSL https://install.pi-hole.net | bash
```

Pi-hole defaults to port 80 for its admin interface. Since Pelican Panel also requires web ports, Pi-hole was configured to avoid conflicts. Once running, local network DNS was pointed to the WSL2 host IP to route all DNS queries through Pi-hole.

Key Outcomes

- 7.4 million+ domains blocked at the DNS level
- Accessible via web admin interface on the local network
- Running stably alongside Pelican Panel and Wings with no port conflicts

4. Pelican Panel Installation

Pelican Panel is an open source game server management panel, forked from Pterodactyl. It provides a web UI for creating, configuring, and monitoring containerized game servers. Installation followed the official documentation at pelican.dev.

Dependencies

- PHP 8.3 with extensions: cli, fpm, mysql, mbstring, xml, curl, zip, bcmath, gd
- Nginx (web server), MySQL (database), Redis (queue driver), Composer (PHP package manager)

Windows Port Forwarding

Because Pelican runs inside WSL2 on an internal IP, Windows port forwarding rules are required to expose services on the LAN IP. Run the following in PowerShell as Administrator on the Windows host:

```
netsh interface portproxy add v4tov4 listenport=8080 listenaddress=0.0.0.0 connectport=8080 connectaddress=172.26.11.55
netsh interface portproxy add v4tov4 listenport=8081 listenaddress=0.0.0.0 connectport=8081 connectaddress=172.26.11.55
netsh interface portproxy add v4tov4 listenport=2022 listenaddress=0.0.0.0 connectport=2022 connectaddress=172.26.11.55
```

■ The WSL2 internal IP changes on every Windows reboot. Port forward rules must be updated whenever the IP changes. A startup script is recommended to automate this.

5. The Core Problem: WSL2 + cgroup v2 + io.weight

Symptom

Every attempt to start or reinstall a game server container through Wings failed with the following error:

```
OCI runtime create failed: runc create failed: unable to start container process:
error during container init: error setting cgroup config for procHooks process:
openat2 /sys/fs/cgroup/docker/[hash]/io.weight: no such file or directory
```

Root Cause

The WSL2 kernel lists io as an available cgroup v2 controller — but it does not implement the io.weight interface file. This file is used by runc (the container runtime) to set IO weight limits on containers. WSL2 strips this interface out because it has no real block device IO scheduler.

Verification confirmed io appears in cgroup.controllers but the file is never created in container cgroup directories:

```
cat /sys/fs/cgroup/cgroup.controllers
# cpuset cpu io memory hugetlb pids rdma

ls /sys/fs/cgroup/docker/[container-hash]/
# io.latency io.max io.pressure io.stat (NO io.weight)
```

Approaches Attempted

Approach	Result
openat_mode: disabled in Wings config	No effect — issue is inside runc, not Wings
Switch Docker to cgroupfs driver	Changed error path, same underlying error
Delegate=yes in Docker systemd unit	No effect on io.weight availability
Write io to cgroup.subtree_control	Wiped on every Docker restart
blkio-weight: 0 in daemon.json	Not a valid daemon option — broke Docker
overlay2 storage driver	Not available in WSL2 — broke Docker

6. The Fix: Patching runc from Source

The definitive solution is to patch runc so that a missing io.weight file is treated as a non-fatal, ignorable error rather than a container init failure. This requires cloning the runc source, applying a one-line code change, compiling, and replacing the system binary.

Step 1 — Install Build Dependencies

```
sudo apt install golang-go make gcc libseccomp-dev pkg-config -y
```

Step 2 — Clone runc Source (Match Installed Version)

```
runc --version
# runc version 1.3.5
```

```
cd ~
git clone --depth=1 --branch v1.3.5 https://github.com/opencontainers/runc.git
cd runc
```

Step 3 — Locate the io.weight Write

```
find . -name "*.go" | xargs grep -l "io.weight" 2>/dev/null
# vendor/github.com/opencontainers/cgroups/fs2/io.go
```

The problematic code is in `vendor/github.com/opencontainers/cgroups/fs2/io.go`. The original line returns a fatal error if the `io.weight` write fails — regardless of why it failed:

```
// ORIGINAL (line ~68)
if err := cgroups.WriteFile(dirPath, "io.weight", strconv.FormatUint(v, 10)); err != nil {
    return err
}
```

Step 4 — Apply the Patch

The patch changes the behavior so that a "file not found" error from `io.weight` is silently ignored, while any other write error is still returned as fatal:

```
python3 -c "
content = open('vendor/github.com/opencontainers/cgroups/fs2/io.go').read()
old = '\"io.weight\\\", strconv.FormatUint(v, 10)); err != nil {\\n\\t\\t\\t\\treturn err'
new = '\"io.weight\\\", strconv.FormatUint(v, 10)); err != nil {\\n\\t\\t\\t\\tif !os.IsNotExist(err) { return err
}'
open('vendor/github.com/opencontainers/cgroups/fs2/io.go', 'w').write(content.replace(old, new))
print('Done' if old in content else 'Pattern not found')
"
```

Step 5 — Compile and Replace

```
cd ~/runc
make

# Backup original binary
sudo cp /usr/bin/runc /usr/bin/runc.backup

# Replace with patched build
sudo cp ~/runc/runc /usr/bin/runc
runc --version
```

Step 6 — Prevent runc from Being Overwritten by apt

```
sudo apt-mark hold runc
```

Step 7 — Restart Services

```
sudo systemctl restart docker
sudo systemctl restart wings
```

7. Docker Configuration

Native Docker Engine is required — Docker Desktop uses a restricted WSL2 integration that makes the cgroup problem worse. The final working `/etc/docker/daemon.json`:

```
{
  "default-cgroupns-mode": "host",
  "default-ulimits": {},
}
```

```
"features": {
  "containerd-snapshotter": true
},
"exec-opts": ["native.cgroupdriver=cgroupfs"],
"log-driver": "json-file"
}
```

Setting	Purpose
default-cgroupns-mode: host	Shares host cgroup namespace — required for Wings container management
native.cgroupdriver=cgroupfs	Bypasses systemd delegation chain that was blocking io subtree control
containerd-snapshotter: true	Enables containerd snapshot storage for better layer management

8. Node & Wings Configuration

Wings config.yml Key Settings

```
api:
  host: 0.0.0.0
  port: 8081
remote: http://192.168.1.182:8080
docker:
  network:
    name: pelican_nw
    interface: 172.20.0.1
openat_mode: disabled
```

Pelican Panel Node Settings

Setting	Value
IP Address	192.168.1.182 (Windows host IP, not WSL2 IP)
Port	8081
Protocol	HTTP
Display Name	sykeshomeserver
Allocation Bind IP	0.0.0.0 (required — WSL2 cannot bind to 192.168.1.182)

■ Setting allocation IP to 192.168.1.182 causes containers to fail on startup with "cannot assign requested address" — always use 0.0.0.0 for allocations in a WSL2 environment.

9. Game Servers

Minecraft — Paper

Setting	Value
Egg	Paper (Pelican official egg)
Port	25565

Setting	Value
Docker Image	Java 25
Startup Command	java -Xms128M -XX:MaxRAMPercentage=95.0 -jar {{SERVER_JARFILE}}
Version	Latest (auto-updated on reinstall)
Status	■ Running
Connect Address	192.168.1.182:25565

Palworld

Setting	Value
Egg	Palworld (community egg)
Port	8211
Docker Image	ghcr.io/parkervcp/steamcmd:debian
Server Version	v0.7.3.90464
Status	■ Running
Connect Address	192.168.1.182:8211

■ *Palworld dedicated servers require the Steam version of the game. The Xbox/GamePass version does not support third-party dedicated servers. Minecraft Java Edition can run in offline mode but this bypasses authentication.*

10. Known Issues & Limitations

Issue	Notes
Node health globe shows red	Cosmetic UI bug in Pelican Panel — Wings is fully operational. Confirmed via API response.
WSL2 IP changes on reboot	Port forward rules reference 172.26.11.55 which changes on every Windows reboot. Must be updated manually or via startup script.
runc patch not persistent through updates	apt-mark hold runc prevents automatic overwrites. If runc is ever manually updated the patch must be reapplied from source.
Docker Desktop conflict	Docker Desktop WSL2 integration must be disabled. Native Docker Engine is required for cgroup compatibility.
No SSL/HTTPS	Panel and Wings run over HTTP only. SSL would require a domain name and certificate — not configured for local LAN use.

11. Post-Reboot Checklist

After a Windows reboot, the WSL2 environment resets. Follow these steps to restore all services:

Step	Action	Command
1	Open WSL2 terminal (Ubuntu)	
2	Get new WSL2 IP	hostname -I awk '{print \$1}'
3	Update port forwards in PowerShell (Admin) if IP changed	netsh interface portproxy show all
4	Add Windows Firewall rules if needed	New-NetFirewallRule -LocalPort 8081 ...
5	Start Docker	sudo service docker start
6	Start Wings	sudo systemctl start wings
7	Verify panel loads	http://192.168.1.182:8080
8	Start game servers from panel	Console tab → Start

12. Skills Demonstrated

- WSL2 environment configuration and service management
- Self-hosted web application deployment (Pelican Panel, Nginx, MySQL, Redis)
- Docker Engine installation, configuration, and daemon management
- Linux systemd service management
- Root cause analysis of a kernel-level cgroup v2 limitation in WSL2
- Go source code patching and compilation (runc from source)
- Windows networking — port forwarding via netsh portproxy, Windows Firewall rules
- Container runtime debugging (runc, OCI runtime errors)
- Network-wide DNS ad blocking with Pi-hole
- Technical documentation and home lab project write-up